

Vibe Coding 개요

1. Vibe Coding이란?

❖Vibe Coding이란?

- AI 전문가 안드레 카파시(Andrej Karpathy)가 처음 제시한 개념으로, 개발자가 생성형 인공지능의 도움을 받아 자연어 프롬프트를 통해 코드를 작성하는 프로그래밍 방식
- 개발자가 Project, Task 를 LLM에 설명하면 LLM이 프롬프트를 기반으로 코드를 생성하고, 개발자는 결과를 평가하고 LLM에 개선사항을 요청하는 방식의 개발 방법
- 개발자는 코드를 세세하게 관리하지 않고 AI가 제안하는 완성을 자유롭게 수용하며, 코드의 정확성이나 구조보다는 반복적인 실험에 더 집중

❖찬성 vs 반대

- 찬성 : 일반인이라할지라도 소프트웨어 엔지니어링 에 필요한 광범위한 교육과 기술 없이 소프트웨어를 생산할 수 있다.
- 반대 : 책임감이 부족으로 인해 생성된 소프트웨어에 보안 취약점이 발생할 위험이 커짐. 전문 개발자의 평가가 필수

1. Vibe Coding이란?

❖Vibe Coding의 특징

- 자연어 기반 개발
  - 사용자가 영어 혹은 모국어로 작성한 프롬프트를 입력하여 원하는 결과를 얻는 방식
- AI 의존적 코딩
  - 코드의 작동 원리를 완전히 이해하지 않아도 코드를 작성할 수 있음
- 직감적 접근
  - 프로그래밍을 할 때 사전에 엄밀한 논리나 설계를 바탕으로 하지 않고 직감과 느낌에 의존

❖기존 코딩과 Vibe Coding의 비교

구분	기존 코딩	Vibe Coding
개발자의 역할	설계자, 개발자, 디버거	프롬프트 작성, 가이드, 테스터
코드 생성	직접 개발자가 작성	AI가 코드를 자동 생성
전문 지식	높은 프로그래밍 언어 지식이 필요함	상대적으로 낮은 기술적 지식이 필요함
개발 속도	체계적이지만 느림	프로토타입 단계에서 빠름
디버깅	수동 디버깅(코드를 이해해야 함)	AI 피드백 기반 반복적 자동 수정

1. Vibe Coding이란?

❖ Vibe Coding의 등장 배경

- 기술적 요인
  - 대규모 언어 모델(LLM)의 발전 : GPT, Claude, Gemini 등의 코드 생성 능력의 비약적인 발전
  - AI 코딩 전용 도구의 등장 : Cursor AI, Windsurf 등의 등장
- 사회적 요인
  - 개발 진입 장벽 완화에 대한 요구 : 비개발자들의 소프트웨어 개발의 문턱을 낮추고자 하는 시대적 요구
  - 빠른 프로토타이핑 : 스타트업과 개인 개발자의 아이디어 구현 속도 향상 요구

2. Vibe Coding이 기존 개발방법론에 끼치는 영향

❖요구사항 분석 단계

- 기존 방식
  - 상세한 기능 명세서 작성
  - 업무 분석가와 개발자간 소통
- Vibe Coding 적용
  - 자연어로 표현된 요구사항을 직접 프로토타이핑으로 변환
  - 빠른 피드백 루프를 통한 요구사항 정제

❖설계 단계

- 기존 방식
  - 아키텍처 설계 및 상세 설계 문서 작성
  - DB 설계, API 설계 등
- Vibe Coding
  - 개발자가 아키텍처를 제안하고 AI가 수정 제안 또는 AI가 기본 아키텍처 제안
  - 프롬프트를 통한 점진적 설계 개선
  - 설계와 구현의 경계 모호화

2. Vibe Coding이 기존 개발방법론에 끼치는 영향

❖구현 단계

- 기존 방식:
  - 설계서 기반 코드 작성
  - 개발자의 직접적인 코딩
- Vibe Coding 적용:
  - 자연어 지시를 통한 코드 생성
  - AI와의 대화형 개발 프로세스
  - 코드 이해 없이도 기능 구현 가능

❖테스트 단계

- 기존 방식:
  - 단위 테스트, 통합 테스트 작성
  - 수동/자동화 테스트 실행
- Vibe Coding 적용:
  - AI가 테스트 케이스 자동 생성
  - 블랙박스 테스트 중심

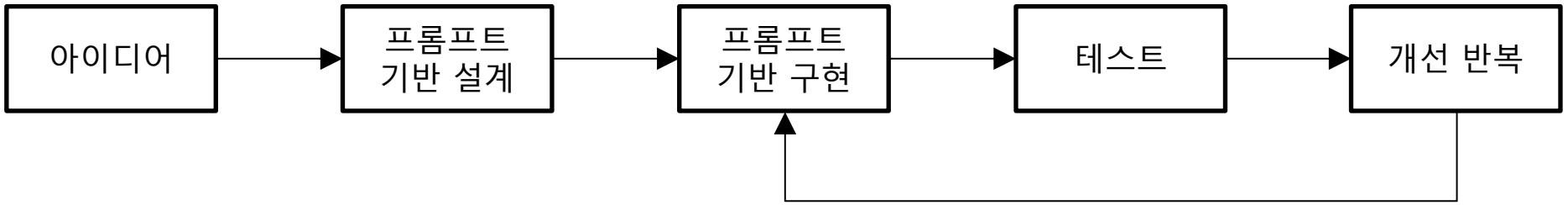
3. VDD(Vibe-Driven Development)

❖전통적 SDLC와의 비교

- 전통적 SDLC 적용



- VDD : 아이디어 -〉 프롬프트 기반 설계 -〉 프롬프트 기반 구현 -〉 테스트 -〉 개선 반복



3. VDD(Vibe-Driven Development)

❖CI/CD 자동화에서의 Vibe Coding

- CI/CD 자동화 개념
  - CI(Continuous Integration):
    - 개발자가 변경한 코드를 중앙 저장소에 자주 통합하고, 빌드와 테스트를 자동 실행하는 프로세스
    - 코드 충돌, 버그를 조기에 발견
  - CD(Continuous Delivery/Deployment):
    - 테스트를 통과한 코드를 운영 환경에 자동으로 배포. 배포 주기 단축, 안정적 릴리즈 보장
  - 자동화된 빌드, 테스트, 배포 파이프라인
    - Jenkins, Circle CI, Gitlab CI, GitHub Actions 와 같은 도구 사용
  - IaC(Infrastructure as Code) : Ansible, Terraform
- Vibe Coding의 특성과 CI/CD
  - Vibe Coding은 CI/CD와 궁합이 매우 좋음
  - 이유
    - 작은 단위의 변경이 많음 : CI/CD로 즉시 빌드·테스트·배포 가능
    - 계속 바뀌는 요구사항 : 자동화로 수동 배포 부담 감소
    - AI 주도 테스트 작성 : 테스트 실패 시 즉시 수정 후 재배포
    - 운영 피드백 반영 속도 향상 : AI 분석 → 코드 수정 → CI/CD로 배포 반복

3. VDD(Vibe-Driven Development)

❖TDD와 Vibe Coding

- TDD : Test Driven Development
  - 테스트 코드를 먼저 작성하고, 해당 테스트를 통과하는 최소한의 코드를 작성하는 개발 방식
  - 목표 : 사전에 정의한 모든 요구사항을 안정적으로 검증
  - 작업 흐름 : Red -〉 Green -〉 Refactor -〉 Red -〉 Green -〉 .....
    - Red : 실패하는 테스트 작성. 아직 기능이 없으므로 테스트 실패
    - Green : 최소한의 코드 작성해 테스트 통과 - 기능 요구를 만족시키는 간단한 구현
    - Refactor : 코드 품질 개선 - 코드 중복 제거, 구조 최적화, 성능 개선
- VDT(Vibe Driven Testing)
  - 프로젝트 진행 중 '개발 흐름(Vibe)'과 AI 도구의 즉각적인 피드백을 활용해 테스트를 작성·수행하는 방식
  - 목표 : 개발자(또는 팀)의 현재 개발 흐름과 맥락에 맞춰, 중요한 부분을 신속하게 검증
  - 작업 흐름 : Vibe -〉 Test -〉 Refine
    - Vibe : 현재 개발 흐름·아이디어를 기반으로 빠른 구현
    - Test : AI 추천/개발자의 직관,경험으로 핵심 기능·위험 지점 테스트 작성 및 실행
    - Refine : AI 피드백 및 테스트 결과 기반 코드 보완

4. AI-DLC

❖AI-DLC란?

- AI를 SDLC의 '중심 동료'로 두고, 모든 개발 단계에 AI를 일관되게 활용하는 방법론.
- 속도와 품질을 동시에 끌어올리는 것이 목표(수십배까지도)

❖두개의 축

- 1) AI 주도 실행 + 인간 감독 (AI가 계획·초안·코드/테스트 생성, 사람은 의사결정)
- 2) 동적 팀 협업(실시간 문제 해결 및 빠른 의사 결정)

❖핵심 작업 흐름

- 계획 수립 → 맥락 질문 → 인간승인 → 구현/검증 → 반복

❖역할 분담

- 사람 : 목표, 제약 정의, 품질/보안에 대한 책임
- AI : 세부 작업 수행, 문서, 테스트, 리팩터링 자동화

❖성공을 위한 권장 사항

- 업무 단위를 AI-친화적 형태로 분해
- 승인 게이트, 보안/라이센스 가드레일, 관측성(로깅, 지표 수집)을 함께 설계

5. Vibe Coding의 실무 적용 범위

❖적합한 프로젝트 vs 적합하지 않은 프로젝트

- 적합한 프로젝트
  - 프로토타입 개발 : 빠른 아이디어 검증
  - 개인 프로젝트 : 1인 개발자의 생산성 향상
- 적합하지 않은 프로젝트
  - 미션 크리티컬 애플리케이션 : 은행, 의료, 항공 시스템
  - 대규모 엔터프라이즈 애플리케이션 : 복잡한 비즈니스 로직
  - 보안이 중요한 애플리케이션 : 금융, 개인정보 관련
- 그렇다면 적합하지 않은 프로젝트에서는 사용할 수 없는가?
  - "전혀 쓰면 안 된다" (X)
  - 핵심 구현·검증 단계에서는 "Vibe Coding에만 의존하는 것을 피해야 한다" (O)
- 하이브리드 접근법
  - 핵심·규제·보안·다중 협업 영역 : TDD, 정형 분석, 코드 리뷰 기반 개발 필수
  - 창의·실험·신속 반복 영역 : Vibe Coding 병행 가능
  - 현실적인 방법 : "VDT + TDD + 코드리뷰"를 혼합해, 빠른 실험과 안전성을 동시에 확보

6. Vibe Coding의 장점과 한계

❖장점

- 생산성 향상
  - 누구나 쉽게 소프트웨어를 만들 수 있다는 점
  - 프로토타입 제작 시간 대폭 단축
  - 반복적 작업 자동화
- 접근성 향상
  - 소프트웨어 개발의 진입 장벽을 낮춤
  - 비개발자도 간단한 앱 제작 가능
  - 창업자, 디자이너의 아이디어 구현 지원
- 창의성 촉진
  - 기술적 제약에서 벗어난 아이디어 중심 개발
  - 다양한 실험과 시도 가능

6. Vibe Coding의 장점과 한계

❖한계와 위험성

- 기술적 한계
  - AI가 생성한 코드에 보안 결함이 의도치 않게 도입될 수 있음
  - 복잡한 비즈니스 로직 구현의 한계
  - 성능 최적화의 어려움
- 개발자 역량 이슈
  - 개발자가 생각하지 않고 코드를 작성하는 습관을 만들 수 있음
  - 기본적인 프로그래밍 스킬 저하 우려
  - 디버깅 능력 감소
- 유지보수 문제
  - 사용자가 AI에 크게 의존하고 생성된 코드에 대한 깊은 이해가 부족한 경우 나중에 버그를 식별하고 수정하는 것이 매우 어려울 수 있음
  - 코드의 빠른 노후화 : AI가 생성한 코드는 작성 당시에는 잘 작동하지만 시간이 지나면서 문제가 발생할 수 있음
    - 최신 라이브러리나 프레임워크 버전과 호환성, 보안 취약점 문제
    - 코드 작성 맥락을 아는 사람이 없어 수정이 어려움
  - 팀 인수인계 어려움

7. 기존 개발자에게 요구되는 것은?

❖AI에게 지시를 내릴 수 있는 아키텍처, 컨텍스트를 이해하는 것

- 시스템 아키텍처 설계
- 성능 최적화
- 보안 설계
- 복잡한 알고리즘 개발
- 프롬프트 예시 차이
  - 예시 1) : "프론트엔드 앱을 위한 JWT를 이용한 인증 기능 서비스를 만들어줘. 서명 알고리즘은 RS-512를 적용하도록 해. 사용자 패스워드는 일방향 해시로 저장하도록 구현해야 해. 사용자 정보는 userid, password, email, phone, ... 을 입력받아 등록할 수 있는 기능을 만들어. 이를 위해 DB 테이블을 PostgreSQL에 생성해야 해"
  - 예시 2) : "프론트엔드 앱을 위한 인증기능 서비스를 만들어 줘"

❖비즈니스 역량 강화

- 요구사항 분석 능력
- 프로젝트 관리 역량
- 비즈니스 도메인 전문 지식

7. 기존 개발자에게 요구되는 것은?

❖개발자의 진화 방향성

- 과거의 개발자: Code Writer
  - 코드를 직접 작성하는 기술자
  - 문법과 알고리즘 전문가
  - 혼자서 문제를 해결하는 능력이 중요
- 현재의 개발자: Problem Solver
  - 비즈니스 문제를 기술로 해결하는 능력이 필요
  - 팀워크와 커뮤니케이션이 중요
  - 다양한 도구를 활용하는 능력이 중요
- 미래의 개발자: AI Collaborator
  - AI와 효과적으로 협업하는 능력이 중요
  - 자연어로 요구사항을 정확히 표현할 수 있어야 함
  - 결과물의 품질과 적절성을 판단하는 안목과 기술적 지식 수준이 요구됨

7. 기존 개발자에게 요구되는 것은?

❖핵심 정리
- Vibe Coding의 장점을 활용하되, 전통적 개발 역량도 유지
- AI의 도움을 받되, 비판적 사고와 품질 의식은 포기하지 않기
- 빠른 프로토타이핑에는 Vibe Coding, 중요한 시스템에는 전통적 방법 적용

❖미래의 성공하는 개발자
- Vibe Coding도 할 수 있고, 전통적 개발도 할 수 있는 하이브리드 개발자

❖AI와의 개발은 '매번 새로운 주니어 개발자 동료와 함께 개발하는 것'
- '새로운 동료'의 의미
  - 프롬프트를 입력하더라도 AI는 기억하지 못한다는 생각으로 매번 반복적으로 AI 에게 알려주어야 함
  - 규칙, 지침, 문서화가 중요함
- '주니어 개발자'의 의미
  - 시키는 것은 잘하지만 창의적으로 작업을 수행할 때는 문제가 발생함
  - 지침을 통해 AI에게 오버엔지니어링, 창의적인 작업을 하지 못하도록 제한해야 함

8. Vibe Coding의 주요 구성요소

❖AI -Native 사고방식

- 기존의 "코드를 직접 작성"하는 방식에서 "AI와 대화하며 개발"하는 패러다임 전환
- 자연어로 의도를 표현하는 능력
- AI의 강점과 한계를 이해하고 적절히 활용
- 코드 작성보다는 문제 정의와 방향 제시에 집중

❖AI 개발도구

- 구현 도구 : 이 과정에서는 Claude Code와 Cursor 도구를 사용함
  - Chat 기반 : Claude, ChatGPT, Grok, Deepseek, Copilot
  - IDE기반 : Cursor IDE, Windsurf
  - Coding Agent 기반 : Claude Code, Gemini CLI, Codex CLI
- 가드레일과 규칙/지침
  - Cursor Rules : 개발 스타일과 아키텍처 가이드라인 정의
  - CLAUDE.md : 프로젝트별 AI 컨텍스트와 규칙
  - Custom Instructions : 개인/팀 개발 패턴 학습

8. Vibe Coding의 주요 구성요소

❖자동화, 운영 인프라

- MCP : Model Context Protocol
  - Smithery : MCP 허브 서비스
  - Database MCP: Supabase 직접 조작
  - Deployment MCP: Vercel 자동 배포
  - Git MCP: GitHub 자동 커밋/푸시
  - API MCP: 외부 서비스 연동
- CI/CD 파이프라인 : 빌드, 테스트 , 배포 자동화
  - GitHub Actions
  - Jenkins, Gitlab CI, Circle CI 등
- 관찰성(Observability)
  - 로그 : ELK 스택 등
  - 지표 : Prometheus 등

8. Vibe Coding의 주요 구성요소

❖표준화된 개발 프로세스

- 1단계: 기획 문서 생성
  - PRD (Product Requirement Document)
  - 사용자 페르소나 및 시나리오
  - 와이어프레임/목업
- 2단계: 아키텍처 설계
  - ERD (Entity Relationship Diagram)
  - API 명세서 (Swagger/OpenAPI)
  - 컴포넌트 구조 정의
  - 디렉토리 구조 정의
- 3단계: 구현
  - 백엔드 API 개발 (프롬프트 기반)
  - 프론트엔드 컴포넌트 개발
  - 실시간 테스트 및 디버깅
- 4단계: 품질 보증
  - 단위 테스트 자동 생성
  - E2E 테스트 (Playwright/Cypress)
  - 보안 검토 (AI 기반)
  - 코드 리팩토링
- 5단계: 배포 및 운영
  - CI/CD 파이프라인 구성
  - 자동 배포 및 모니터링

8. Vibe Coding의 주요 구성요소

❖품질관리 도구

- 테스팅 도구
  - Vitest, Jest : 단위 테스트
  - Playwright, Cypress : E2E 테스트
- 코드 품질 도구
  - ESLint: 코드 스타일 검사
  - Prettier: 코드 포매팅
  - TypeScript: 타입 안전성

❖프롬프트 엔지니어링

- 프로젝트 문서를 AI 컨텍스트로 활용
- 점진적 개발을 위한 이전 결과물 참조
- 일관성 유지를 위한 스타일 가이드 적용

9. 정리

❖Vibe Coding 개념

- 자연어 프롬프트로 LLM이 코드를 만들고, 개발자는 평가·지시를 반복하는 대화형 개발 방식.

❖프로세스 변화

- 요구분석~테스트 전 단계에서 프로토타이핑 가속, 설계/구현 경계 희미화, AI 기반 테스트·리팩터링 보조.

❖방법론(VDD/VDT)

- 아이디어→프롬프트→구현→테스트→개선의 빠른 루프와 흐름 중심 테스트로 신속 검증.

❖장점/한계

- 생산성과 접근성은 높지만 보안·책임·복잡 로직/성능 최적화·유지보수 리스크가 존재.

❖AI-DLC 포인트

- SDLC 전 과정에 AI를 '중심 동료'로 두고 계획→질문→승인→구현/검증의 루프, 인간은 품질·보안 책임.

❖적용과 역할

- 프로토타입·개인 프로젝트에 적합, 핵심 시스템은 TDD·코드리뷰와 병행 권장; 개발자는 요구정의·품질판단·프롬프트 설계 역량이 핵심.

10. SDD(Spec Driven Development)

❖SDD란?

- 코드를 작성하기 전에 시스템의 행동과 기능을 정의하는 상세한 명세서(Specification)를 먼저 작성하는 소프트웨어 개발 방법론
- 명세 : 단일 진실 공급원(Single Source of Truth)

❖특징

- 명세 우선 : 시스템의 행동, 속성을 기술하는데 중점을 둠
- AI 협업 중심 : 명세를 이용해 AI가 개발을 진행함
- 명세 문서 유지 : 명세는 개발이 완료된 후 버려지는 것이 아닌 지속적으로 관리됨
- 명세 중심의 의사소통 : 상세한 계획과 작성된 문서가 의사소통의 기본 수단, 기본 소스임

❖SDD 작업 방식

- Steering --〉 Spec --〉 Implement
- Steering : 리포지토리 루트의 지침. Spec, Code와 관련된 일관된 지침을 포함시킴
- Specs : 요구사항, 설계 문서를 생성하고, 순차적으로 실행할 개발 단계를 작업을 정의함
- Implement : 작성,생성된 Spec을 기반으로 AI가 코드를 구현

10. SDD(Spec Driven Development)

❖SDD의 장점

- 의사결정 명문화
  - 명세가 코드 변경의 "근거"가 되어 리뷰·합의가 쉬워짐.
- 조합 가능한 살아있는 스펙
  - 사용자 시나리오·수용 기준·인터페이스/데이터 계약·성능/SLAs 등을 모듈화해 재사용·합성 가능
- SDLC 전반 적용
  - 요구·설계 앞단 정렬, 배포 후 이슈의 피드백 루프까지 AI 시대 코드 생성 속도 속에서도 리뷰/품질/일관성을 챙기자는 문제의식.

❖Spec kit : https://github.com/github/spec-kit

- 제품 시나리오와 예측 가능한 결과에 집중할 수 있게 해주는 SDD 기반 오픈 소스 툴킷
- 개발 흐름
  - Specify : 프로젝트 원칙(헌장) 수립 + Spec 작성
  - Plan : 기술 스택, 애플리케이션 아키텍처, 제약 조건 포함
  - Tasks : 계획 수립 결과를 바탕으로 작고 테스트 가능한 단위로 분할함
  - Implement : 설계, 계획을 바탕으로 구현, 테스트함